

Vapi configuration

Everything the Vapi dashboard needs so the assistant and this repo agree.

The architecture decision comes first

Vapi can be wired to a backend in two ways, and they are mutually exclusive.

	CUSTOM LLM (WHAT THIS REPO IMPLEMENTS)	CUSTOM TOOL
Who reasons	Our agent, via <code>/api/vapi/chat/completions</code>	Vapi's own LLM
Who holds the system prompt	<code>packages/agent/src/system-prompt.ts</code>	The Vapi dashboard
Who calls the MCP tools	Our agent	Our agent, behind a <code>support_agent</code> tool
Dashboard prompt is	Ignored — the route drops Vapi's system message	The whole brain
Extra work needed	None	A <code>support_agent</code> tool endpoint, which does not exist yet

This repo implements Custom LLM. `docs/vapi-system-prompt.md` was written for the *custom tool* shape. Pasting it into an assistant that has no `support_agent` tool is what produces a call where the agent answers "what are you?" normally but says *"I can't reach our support system"* for every real question: the prompt orders it to call a tool it does not have, and the quoted sentence is the prompt's own failure line.

Pick Custom LLM unless you specifically want Vapi's LLM handling small talk.

1. Model

Assistant → Model

FIELD	VALUE
Provider	Custom LLM
URL	<code>https://<your-host>/api/vapi</code>
Secret / API key	the same string as <code>VAPISERVERSECRET</code> in your env
Model name	anything (e.g. <code>relaypay-support-agent</code>) — our route ignores it
System prompt	leave empty or one line: the route discards it

On the URL: **Vapi appends /chat/completions to whatever you type.** The dashboard says so directly under the field — it previews the full URL it will call. So enter the base and let it append:

YOU TYPE	VAPI CALLS	RESULT
<code>https://<host>/api/vapi</code>	<code>https://<host>/api/vapi/chat/completions</code>	correct
<code>https://<host>/api/vapi/chat/completions</code>	<code>.../chat/completions/chat/completions</code>	404 every turn
<code>https://<host>/chat/completions</code>	<code>/chat/completions/chat/completions</code>	404 every turn

Read that preview line before saving; it is the fastest way to catch this. The repo also serves the handler at `/api/vapi` itself, so a GET to the base returns a health payload you can check in a browser.

The system prompt genuinely does not matter here. The route drops Vapi's system message on purpose and uses the agent's own, which carries the security rules. Changing the dashboard prompt will not change the agent's behaviour — edit `packages/agent/src/system-prompt.ts` instead.

IF CALLS DROP WITH "EJECTED"

Vapi ends the room when the model does not answer in time. Its custom-LLM `timeoutSeconds` is documented as the timeout "without needing to stream any tokens back" and defaults to **20 seconds** — shorter than an agent turn, which measured 24.9s on the deployed service.

This route opens its SSE response immediately (first byte in under 0.1s) and fills it when the agent is done, so the timeout stops applying and no configuration change is needed. If you still see drops, raise `model.timeoutSeconds` on the assistant — the API accepts up to 300:

```
curl -X PATCH https://api.vapi.ai/assistant/<assistant-id> -H "Authorization: Bearer <your-private-key>" -H "Content-Type: application/json" -d '{"model":{"provider":"custom-llm","url":"https://<host>/api/vapi","timeoutSeconds":120}}'
```

2. Server URL (end-of-call)

Assistant → Advanced → Server URL

FIELD	VALUE
Server URL	<code>https://<your-host>/api/vapi/end-of-call</code>
Secret	the same <code>VAPISERVERSECRET</code>

This is what persists the transcript for admin review. Without it, calls happen but nothing is saved.

THE TWO SECRET FIELDS ARE SEPARATE

Vapi authenticates the model endpoint and the Server URL independently, and they are configured in different places. Both must carry the same string as `VAPISERVERSECRET` on the server.

LOG LINE	WHAT TO FIX
<code>secret mismatch (values sent via authorization)</code>	The custom-llm credential holds the wrong value — see below.

RELAYPAY SUPPORT AGENT • EMMANUEL ABOYEJI • WEEK 6 • 2026-10-03

LOG LINE	WHAT TO FIX
[end-of-call] ... secret header present but empty	Advanced → Server URL Secret is blank.
[end-of-call] ... no auth headers at all	Vapi is sending nothing; the Server URL has no secret configured.

THE AUTHORIZATION HEADER COMES FROM A CREDENTIAL, NOT THE ASSISTANT

Vapi's schema is explicit: custom headers "can override default OpenAI headers **except for Authorization (which should be specified using a custom-llm credential)**". So the value arriving in `authorization` comes from a stored credential in your Vapi account, not from a field on the assistant. Editing the assistant will not change it.

Two ways to make it match `VAPISERVERSECRET` :

Set the credential — in the Vapi dashboard, find the custom-llm provider credential the assistant uses and set its API key to the secret.

Or send it as a custom header instead, which avoids the credential entirely. This endpoint accepts `x-vapi-secret` , and `headers` *can* set that one:

```
curl -X PATCH https://api.vapi.ai/assistant/<assistant-id> -H "Authorization: Bearer <your-vapi-private-key>" -H "Content-Type: application/json" -d '{
  "model": {
    "provider": "custom-llm",
    "model": "relaypay-support-agent",
    "url": "https://<your-host>/api/vapi",
    "timeoutSeconds": 120,
    "headers": { "x-vapi-secret": "<your VAPI_SERVER_SECRET>" }
  }
}'
```

A PATCH replaces the whole `model` object, so include every field you want kept.

A burst of `end-of-call` rejections per call is normal when the secret is missing: Vapi posts several message types (status updates, transcripts, end-of-call-report) to the Server URL, and every one is rejected. Narrowing the assistant's `serverMessages` to `end-of-call-report` removes the noise — nothing else is used here.

3. First message

Set **First message** to something short, or leave Vapi to ask our endpoint. Our route already returns a greeting when there is no user turn yet:

Hi, thanks for calling RelayPay support. This call may be recorded for quality and support purposes. How can I help today?

Set it in one place only, or the caller hears two greetings.

4. Local development

Vapi calls your server from its cloud, so `localhost` is unreachable. Expose it:

```
npx ngrok http 3000
RELAYPAY SUPPORT AGENT • EMMANUEL ABOYEJI • WEEK 6 • 2026-10-03
```

Then set the Model URL to `https://<id>.ngrok.app/api/vapi` and the Server URL to `https://<id>.ngrok.app/api/vapi/end-of-call` . The free tier changes the subdomain on every restart, so both have to be updated each time.

5. Checking it without making a call

```
curl -X POST http://localhost:3000/api/vapi \
-H 'content-type: application/json' \
-H 'x-vapi-secret: YOUR_VAPI_SERVER_SECRET' \
-d '{"model": "x", "stream": true, "call": {"id": "t1"},
  "messages": [{"role": "user", "content": "What is the status of TXN-9001?"}]}'
```

Expected: `data:` lines ending in `data: [DONE]` . What you get tells you where the problem is:

RESPONSE	MEANING
401 Unauthorized	VAPISERVERSECRET missing or different from the header
404	wrong path — use <code>/api/vapi</code>
A real answer	backend is healthy; any remaining fault is in the Vapi dashboard
"I'm having trouble processing that right now"	the agent ran but a tool failed — check the server log for <code>[agent]</code>

6. Prerequisites that are easy to miss

- **Run all four migrations** in `supabase/migrations/` against the project `SUPABASEURL` *points at, then* `npm run seed` . If `Lookup` `transaction` cannot find `TXN-9001` , the schema or the seed is missing — or `SUPABASE_URL` is pointing at a different project.
- `npm run build` **before** `npm run dev` . The agent spawns `packages/mcp-server/dist/index.js` ; if it was never built, every tool-using turn fails.
- **Node 20 or later.** On Node < 22 the Supabase client needs a WebSocket implementation, which `packages/shared` now supplies via `ws` .